Outside-In Review

# jurassic-code

Analyzed 2026-03-09 · ~1h time-box · commit 3bd6882 · Skill by **Yoan Thirion**

---

**CONTENTS**


- **Report**
- **Architecture**
- **Flow: move a dinosaur**
- **Backlog**
- **Code quality**

# Dashboard




VERDICT

## At risk

A small, readable teaching codebase whose backend runs on an EOL framework, shares one global mutable store across every request, and has no CI to catch regressions. The React frontend is modern; the .NET backend is not.

01 · DOCUMENTATION

**Health** 40/100

1 critical · 3 warnings

02 · CI/CD

**Health** 20/100

2 critical · 2 warnings

03 · GIT HISTORY

**Health** 80/100

0 critical · 2 warnings

04 · BUILD

**Health** 70/100

0 critical · 3 warnings

05 · WARNINGS

**Health** 20/100

2 critical · 2 warnings

06 · STRUCTURE

**Health** 0/100

2 critical · 4 warnings

07 · DEPENDENCIES

**Health** 30/100

2 critical · 1 warning

08 · FRESHNESS

**Health** 30/100

2 critical · 1 warning

09 · QUALITY METRICS

**Health** 20/100

2 critical · 2 warnings

10 · HOTSPOTS

**Health** 20/100

2 critical · 2 warnings

11 · ACCESSIBILITY

**Health** 60/100

0 critical · 4 warnings

12 · ECO-DESIGN

**Health** 40/100

1 critical · 3 warnings

CODE QUALITY

**F**

14 smells: 5 critical, 5 significant, 4 minor

ARCHITECTURE

**4/4**

C4 levels rendered and validated (5/5 diagrams in the report)

DETAILED FLOW

**Move**

Move a dinosaur between zones, UI to in-memory store

BACKLOG

**9**

features in 5 domains, 6 with a test, 4 incomplete or defective

## 01 Documentation CRITICAL

The README presents the project as a workshop for DevOps Days Geneva 2025 ("taming the dinosaurs of legacy code") and links to a Miro board and a cards PDF. It gives no instructions to build, run or test the application: a newcomer has to read the source tree to discover there is an API and a React frontend.

CRITICAL **No setup instructions.** The README only holds workshop material; no build, run or test steps.

WARNING **No architecture or decision records.** `docs/` holds workshop cards and AI-generated artifacts, not living technical documentation.

WARNING **Undocumented prerequisites.** .NET SDK version, Node/npm versions and `dotnet-stryker` are never mentioned.

WARNING **Undocumented scripts.** `src/run.sh`, `src/stop.sh` and `src/build.sh` exist without any explanation.

OK **Swagger UI** is served at the API root (`Startup.cs:65`), so endpoints document themselves once the server runs.

## 02 CI/CD Pipelines CRITICAL

There is no pipeline on the current branch. `src/.github/workflows/ci.yml` appears in five commits of the history but is gone from the working tree. Nothing gates a merge on `main`.

CRITICAL **No active CI/CD pipeline.** No `.github/workflows/` directory on the current branch.

CRITICAL **No quality gates.** No coverage threshold, static analysis or mutation testing runs automatically.

WARNING **CI existed once.** The workflow file is in the history, so the intent was there and was dropped.

WARNING **No PR protection.** Merges to `main` are unguarded.

---

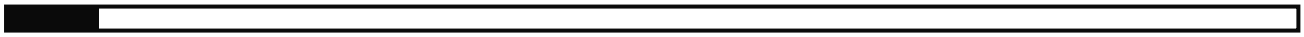
## 03 Git History WARNING

67 commits over about four months (2025-02-24 to 2025-06-30), by two contributors. The most changed files are workshop material ( `IDEAS.md` , `claude/steps.md` ) rather than production code. Commit messages follow Conventional Commits consistently.

**Yoan Thirion — share of commits** 62 · 93%



**Alexandre Trigueros — share of commits** 5 · 7%



WARNING **Near-single contributor.** One person owns 93% of commits; knowledge is concentrated.

WARNING **Most changed files are docs and workshop artifacts.** The code itself is treated as a static teaching fixture, not an evolving product.

OK **Conventional commits.** `feat:` , `fix:` , `refactor:` , `docs:` used throughout.

OK **Active repository** at the time of analysis (last commit June 2025).

---

## 04 Build & Compilation WARNING

`dotnet build JurassicCode.sln` succeeds without extra steps and the 3 tests pass. All four .NET projects target `netcoreapp3.1`, end of life since December 2022. The Vite/React frontend has no documented start command and proxies `/api` to `https://localhost:5001` (`vite.config.ts`).

- WARNING **EOL target framework.** `netcoreapp3.1` receives no security patches.
- WARNING **No Docker Compose or dev container.** API and frontend must be started by hand.
- WARNING **Frontend setup undocumented.** No `npm install` / `npm run dev` instructions.
- OK **Clean build from a single command** in about 4 seconds.
- OK **All 3 tests pass** on the current state.

---

## 05 Compilation Warnings CRITICAL

The build emits 14 warnings. Two matter: a known vulnerability in `Swashbuckle.AspNetCore.SwaggerUI 5.6.3` and `Polly 2.2.0` restored against .NET Framework 4.x targets. The EOL framework warning is raised on every project.

**Projects on an EOL framework** 4/4

- CRITICAL **Vulnerable Swagger UI.** `Swashbuckle.AspNetCore.SwaggerUI 5.6.3` is flagged by advisory `GHSA-qrmm-w75w-3wpx` (moderate).
- CRITICAL `netcoreapp3.1` **EOL on every .NET project.**
- WARNING **Polly 2.2.0 incompatible with the target.** NuGet resolved it against .NET Framework 4.x.
- WARNING **14 warnings in a 4-project solution.** None suppressed, but enough noise to hide real problems.

---

## 06 Code Structure CRITICAL

Five projects, no discernible architecture. The domain library is flat; the service is a `partial class` split between `Class1.cs` and `Init.cs`; persistence goes through a static class holding a public static database; and `Database` reads its own private field by reflection.

```
src/
├── JurassicCode/            domain + persistence, flat
│   ├── Class1.cs            ParkService (partial) + IParkService   ← hotspot
│   ├── Init.cs              ParkService (partial): 277 lines of seed data
│   ├── ReflectionHelper.cs  reads private fields
│   └── Db2/                 DataAccessLayer (static), Database, Entities
├── JurassicCode.API/        ASP.NET Core: ParkController, WeatherForecastController (temp
├── JurassicCode.Tests/      xUnit: 3 tests
├── JurrassicCode.Console/   VB.NET client (typo in the name), duplicates the seed data
└── jurassic-ui/             React 19 + Vite + styled-components
```

CRITICAL **No discernible architecture.** No layering, ports/adapters or feature modules.

CRITICAL **Business logic coupled to static data access.** `ParkService` reads and mutates `DataAccessLayer._db` directly ( `Class1.cs:29-196` ).

WARNING `Class1.cs` keeps the default IDE name for the central service.

WARNING **Partial class split** across `Class1.cs` and `Init.cs` for no structural reason.

WARNING **Second runtime client** ( `JurrassicCode.Console` , VB.NET) duplicates the seed data.

WARNING **Inconsistent names across the domain/persistence boundary.** `IsCarnivorous` ↔ `IsVegan` (inverted), `Name` ↔ `CodeName` , `IsOpen` ↔ `AccessStatus` .

---

## 07 Dependencies CRITICAL

The backend declares four third-party packages; two of them are never used and one carries a known advisory. The frontend dependency set is modern and lean.

### Backend (C#)

| Package | Version | Used? | Status |
|---|---|---|---|
| `Swashbuckle.AspNetCore` | 5.6.3 | yes | ADVISORY |
| `Swashbuckle.AspNetCore.SwaggerUI` | 5.6.3 | yes | ADVISORY |
| `Confluent.Kafka` | 0.11.1 | no | ABANDONED MAJOR |
| `Polly` | 2.2.0 | no | INCOMPATIBLE |

### Frontend (TypeScript / React)

| Package | Version | Status |
|---|---|---|
| `react` | ^19.0.0 | CURRENT |
| `vite` | ^6.2.0 | CURRENT |
| `react-router-dom` | ^7.3.0 | CURRENT |
| `styled-components` | ^6.1.15 | MAINTAINED |
| `axios` | ^1.8.2 | DECLARED, UNUSED (FETCH IS USED) |

CRITICAL **`Confluent.Kafka 0.11.1`**: a 2018 pre-1.0 release, never imported.

CRITICAL **`Polly 2.2.0`**: current is 8.x; 2.x dates from 2016 and does not fit the runtime.

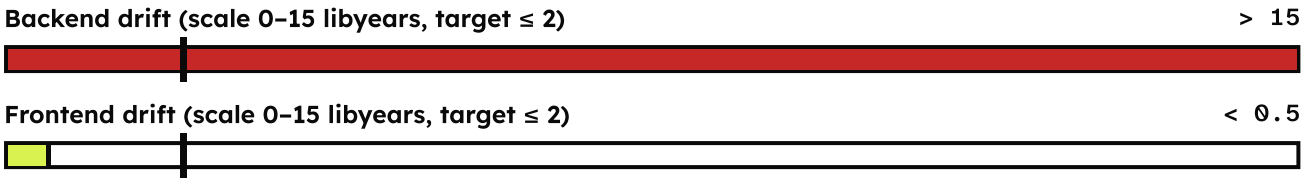
WARNING **Dead dependencies.** Kafka, Polly (backend) and axios (frontend, `api.ts` uses `fetch`) are declared but unused.

OK **Frontend dependencies are modern and maintained.**

---

## 08 Dependency Freshness CRITICAL

Backend drift exceeds 15 libyears: Kafka is about 8 years behind, Polly about 6, Swashbuckle about 3, on top of the EOL runtime. The frontend sits under half a libyear.




CRITICAL **> 15 libyears of backend drift**, concentrated in unused packages and the EOL runtime.

CRITICAL **HTTP-facing middleware outdated** (Swashbuckle, with a known advisory).

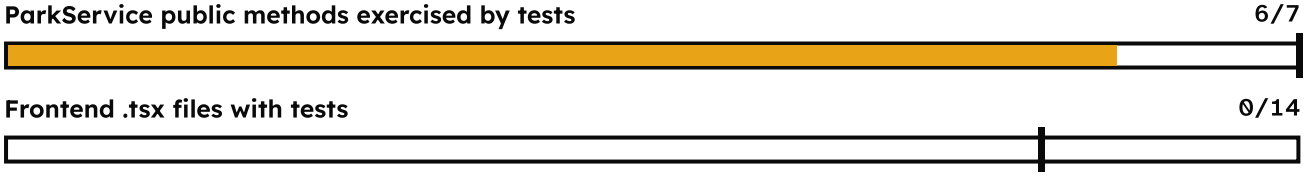
WARNING **Selective maintenance.** The frontend is kept current while the backend rots.

OK **Frontend drift under 0.5 libyears.**

---

## 09 Quality Metrics CRITICAL

Three xUnit tests, all passing. No coverage tool is configured. Stryker is configured (`stryker-config.json`) but no mutation score was ever recorded. The tests assert counts and exceptions, not the returned data, which is how a real defect slips through (see **Code quality**).




NOT MEASURED Line coverage · NOT MEASURED Mutation score

CRITICAL **Coverage estimated under 40%** across the solution: 3 tests, no test on the API, the console client or the frontend.

CRITICAL **No mutation baseline.** Stryker is configured but never run in CI.

WARNING `CanSpeciesCoexist` **hardcodes 3 species** (`Class1.cs:126-137`); any other pair returns `true`.

WARNING **Tests share global static state.** They reset `DataAccessLayer._db` with `DataAccessLayer.Init(new Database())`.

OK **Mutation tooling is present**, only enforcement is missing.

## 10 Hotspots CRITICAL

Change frequency crossed with structural complexity. The service file concentrates the risk; the controller multiplies the same error handling seven times. One author owns every production file.

| File | Complexity | Commits | Primary author |
|---|---|---|---|
| `JurassicCode/Class1.cs` | High: 7 methods, index loops, nesting 4, global state | 4 | Yoan Thirion |
| `JurassicCode.API/Controllers/ParkController.cs` | Medium: 7 endpoints, 7 identical catch blocks | 5 | Yoan Thirion |
| `JurassicCode.Tests/Tests.cs` | Medium: shared static state | 5 | Yoan Thirion |
| `JurassicCode/Db2/DataAccessLayer.cs` | Medium: static class, public static field | 2 | Yoan Thirion |
| `JurassicCode/Init.cs` | Low: seed data only, 277 lines | 2 | Yoan Thirion |

**Production files owned by a single author (knowledge islands)** 100%



CRITICAL `Class1.cs`: highest complexity, direct global-state access everywhere, single author.

CRITICAL **Single author on all production code.**

WARNING **Seven copy-pasted catch blocks** in `ParkController`: any change to error handling touches every endpoint.

WARNING `DataAccessLayer` **has no direct test** although it is the most dangerous class.

## 11 Accessibility WARNING

The project is written in English, so **WCAG 2.1 AA** applies to `jurassic-ui`. The basics are right: `lang="en"`, a `<main>` landmark, real `<button>` elements and labelled form fields. The gaps are in the dynamic parts: the move dialog, tabs and focus styling. Static analysis only; contrast was not measured.

**WCAG criteria checked and passed** 4/8

| Criterion | Status | Evidence |
|---|---|---|
| 3.1.1 Language of Page (A) | PASS | `index.html:2` `<html lang="en">` |
| 1.3.1 Info and Relationships (A) | PASS | `Layout.tsx:39` `<main>`; `<Label htmlFor="toZone">` in `MoveDinosaurForm.tsx:82` |
| 2.1.1 Keyboard (A) | PASS | Actions are native `<button>` elements (`DinosaurCard.tsx:52`, styled `Tab`) |
| 1.1.1 Non-text Content (A) | PASS | No `<img>` in components; decorative SVG is `aria-hidden` |
| 4.1.2 Name, Role, Value (A) | FAIL | Close button is a bare `×` without `aria-label` (`MoveDinosaurForm.tsx:112`); tabs lack `role="tab"` / `aria-selected` (`Dinosaurs.tsx:136`) |
| 2.4.3 Focus Order (A) | FAIL | Modal is a `div` with no `role="dialog"`, no `aria-modal`, no focus trap or focus return (`styled/index.ts:334`) |
| 2.4.1 Bypass Blocks (A) | FAIL | No skip link before the navbar (`Layout.tsx`) |
| 2.4.7 Focus Visible (AA) | FAIL | `outline: none` replaced by a 30%-opacity box-shadow on inputs and selects (`styled/index.ts:111,132`) |

WARNING **Inaccessible modal.** Keyboard and screen-reader users can leave the move dialog without knowing it, and focus is not returned.

WARNING **Unnamed close control** ( `×` ) and **unannounced tab state**.

WARNING **No skip link** to main content.

WARNING **Weak focus indicator** after removing the native outline.

OK **Semantic foundations**: page language, landmark, native buttons, associated labels.

---

## 12 Eco-design CRITICAL

English project, so the **Web Sustainability Guidelines (WSG 1.0)** apply. The app always fetches the whole park: every page load, every action and a 30-second polling loop call `GetAllZones`, which returns every zone with every dinosaur and has no pagination. Assets add avoidable requests.

**WSG practices checked and met** 2/6

| Practice | Status | Evidence |
|---|---|---|
| Paginate and scope collection responses | FAIL | `GET /Park/GetAllZones` returns the full graph; called from 8 places across pages ( `Zones.tsx` , `Dinosaurs.tsx` , `Dashboard.tsx` ) |
| Event-driven updates over polling | FAIL | `setInterval(fetchData, 30000)` in `Dashboard.tsx:57` |
| 2.11 Efficient file formats | FAIL | Custom font shipped as `.ttf` ( `public/fonts/jurassic-park.ttf` , 41 KB) instead of WOFF2 |
| Avoid unnecessary requests | FAIL | Full-viewport `/images/jungle-background.jpg` and the favicon are referenced but `public/images/` does not exist: two 404s per load; 4 Roboto weights from Google Fonts on top of the custom font |
| Lean dependency tree | PASS | 5 runtime dependencies (1 unused) |

| Practice | Status | Evidence |
|---|---|---|
| 3.6 Caching of static assets | PASS | Vite emits hashed asset names, cacheable by any static host |

CRITICAL **Unbounded collection endpoint** re-fetched after every action instead of fetching what changed.

WARNING **Polling every 30 seconds** on the dashboard, even when the tab is hidden.

WARNING **Font payload**: TTF instead of WOFF2, plus 4 remote Roboto weights.

WARNING **Requests to missing assets** (background image, favicon).

OK **Small, modern frontend stack** with hashed static assets.

---

## 13 Summary

### Top 3 risks

1. CRITICAL **EOL runtime and vulnerable Swagger UI.** `netcoreapp3.1` gets no security patches and `Swashbuckle 5.6.3` carries `GHSA-qrmm-w75w-3wpx`. A production risk as soon as the API is exposed.
2. CRITICAL **Global mutable static state.** `DataAccessLayer._db` is a public static field shared by every request, mutated without locking, and reset by every `new ParkService()`: the seed data written in `Program.InitializePark()` is wiped when DI creates its own instance.
3. CRITICAL **Wrong data, green tests, no CI.** `GetDinosaursInZone` returns dinosaurs with no species and every one flagged carnivorous, yet the tests pass because they only count items. Nothing runs them automatically.

### Top 3 questions to validate with the team

1. Are `Confluent.Kafka` and `Polly` deliberate workshop "fossils", or leftovers of a planned feature?

2. Is the static `DataAccessLayer` an intentional legacy smell for the workshop, or should the exercise end with proper dependency injection?
3. Was the CI workflow removed on purpose as part of the exercise?

## Recommended next actions

| Immediate | Short term | Medium term |
| --- | --- | --- |
| Upgrade Swashbuckle to 6.x+ and the runtime to .NET 8 LTS. Fix `DataAccessLayer.GetDinosaurs` to return stored entities. | Replace the static `DataAccessLayer` with an injected repository; register `IParkService` once; make the modal accessible. | CI with build, tests, coverage gate (≥ 80%) and Stryker gate. Rename `Class1.cs`, merge the partial class, drop unused packages, scope the zones endpoint. |

## Detailed analysis

ARCHITECTURE

**C4 model**

Context, Container, Component and Code diagrams, with assumptions and rendering check.

FLOW

**Move a dinosaur**

End-to-end sequence, step by step, business rules and risks.

BACKLOG

**9 features**

User stories with Gherkin acceptance criteria and examples.

CODE QUALITY

**Grade F**

14 smells located, explained and paired with a refactoring.

Architecture

# Architecture — jurassic-code

Analyzed 2026-03-09 · C4 model · Skill by **Yoan Thirion**

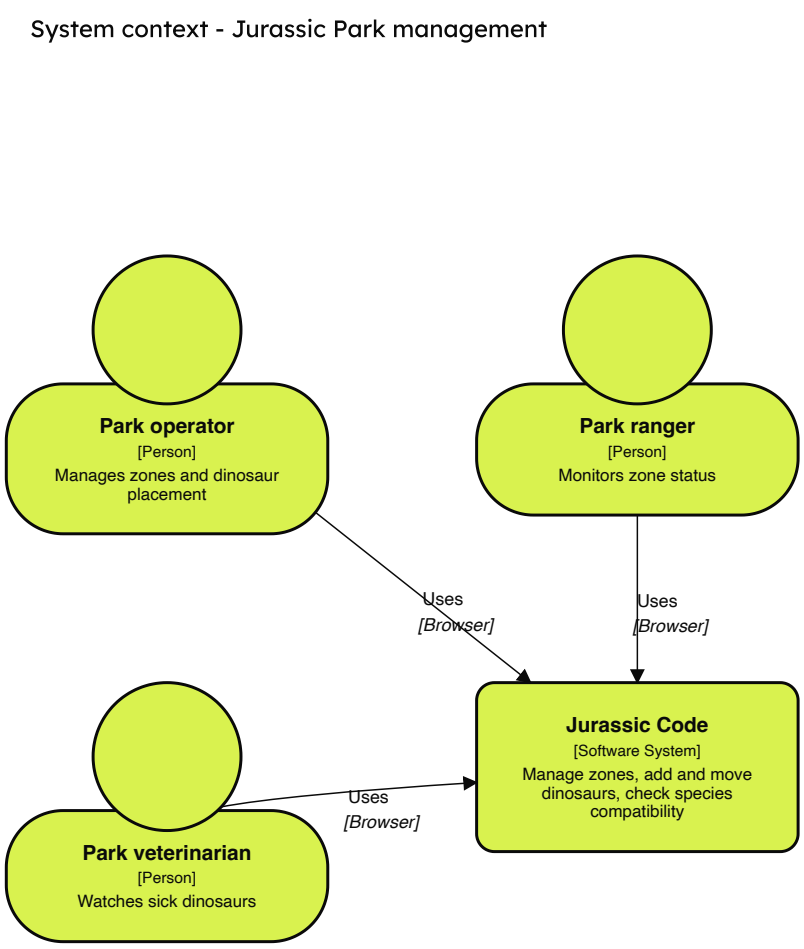
> AI-inferred from code — validate with the team before sharing externally.

## L1 System context

One system, three kinds of park staff, no external system. Everything (UI, API, storage) runs on the developer machine; there is no authentication.

C4 · LEVEL 1 · SYSTEM CONTEXT

RENDERED

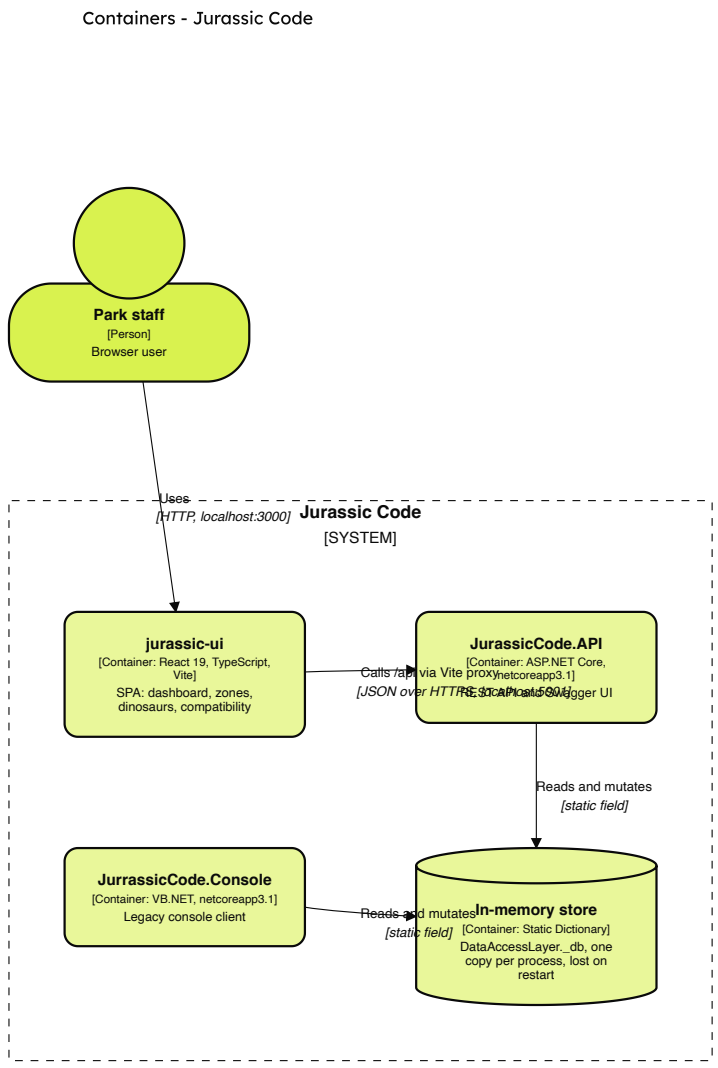


## L2 Containers

A React SPA talks to an ASP.NET Core API through the Vite dev proxy. The "database" is a dictionary held in a static field: the API and the VB.NET console client each get their own copy in their own process, nothing is shared and nothing is persisted.

C4 · LEVEL 2 · CONTAINERS

RENDERED

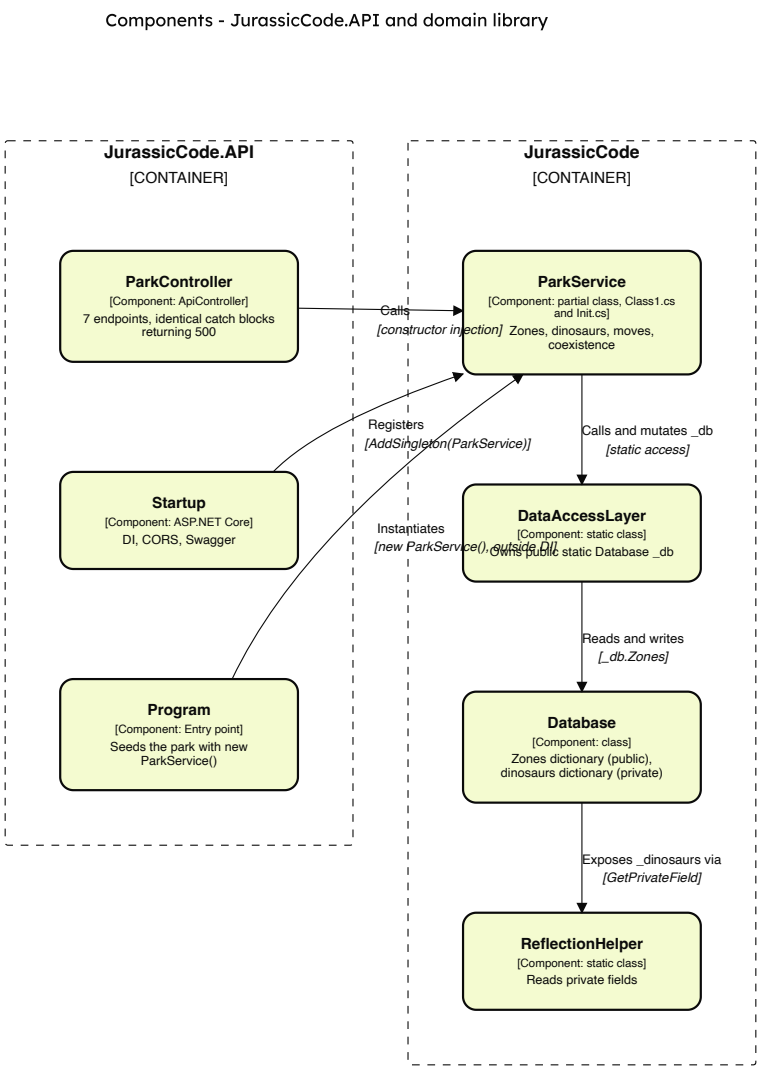


## L3 Components of the API

Read it from left to right: the controller depends on the service through DI, but `Program` also news up its own `ParkService`, and every instance resets the static store. Below the service, encapsulation is bypassed twice: a public static field and reflection on a private field.

C4 · LEVEL 3 · COMPONENTS

RENDERED

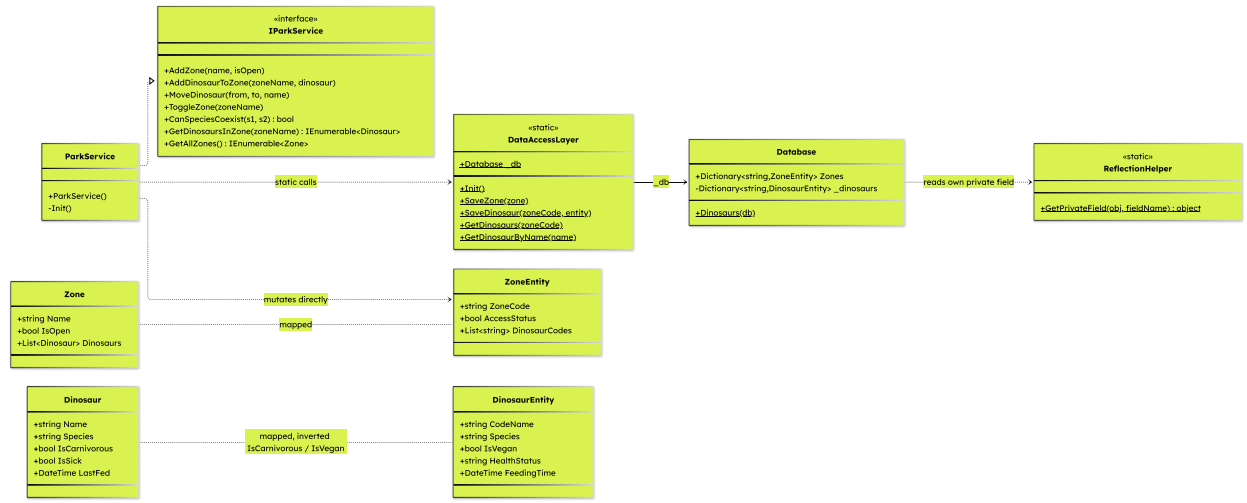


# L4 Code: ParkService and persistence

The domain model ( `Dinosaur` , `Zone` ) and the persistence model ( `DinosaurEntity` , `ZoneEntity` ) carry the same data under different, sometimes inverted, names. `ParkService` cannot be instantiated without touching the static store.

C4 · LEVEL 4 · CODE

RENDERED




> **Key structural problem.** `Database` makes `_dinosaurs` private, then exposes it through `ReflectionHelper.GetPrivateField()` ( `Database.cs:10` ). The class breaks its own contract, and `ParkService` goes around the data layer anyway by mutating `ZoneEntity` instances it reads from `_db` .

---

## Assumptions

- The three roles (operator, ranger, veterinarian) are inferred from the vocabulary of the UI and models; the code has no roles or authentication.
- The console client is assumed to be a legacy tool kept for the workshop; nothing in the API calls it.
- The Vite proxy target `https://localhost:5001` is assumed to be the only supported way to reach the API in development.

---

## Rendering check

5/5 RENDERED

Validated with `scripts/validate_diagrams.py` (headless Chrome, embedded Mermaid 12.3.0).

| Diagram | Type | Status |
| --- | --- | --- |
| **L1 System context** | `C4Context` | RENDERED |
| **L2 Containers** | `C4Container` | RENDERED |
| **L3 Components** | `C4Component` | RENDERED |
| **L4 Code** | `classDiagram` | RENDERED |
| **Flow: move a dinosaur** | `sequenceDiagram` | RENDERED |

Flow

# Move a dinosaur between zones

Analyzed 2026-03-09 · UI → API → domain → in-memory store · Skill by **Yoan Thirion**

---

## Why this flow

Moving a dinosaur is the only use case that crosses every layer (React form, API client, Vite proxy, controller, service, static store) and then refreshes the whole park. It runs through the main hotspot ( `Class1.cs` ) and is where the park's core safety rule, species compatibility, should apply.

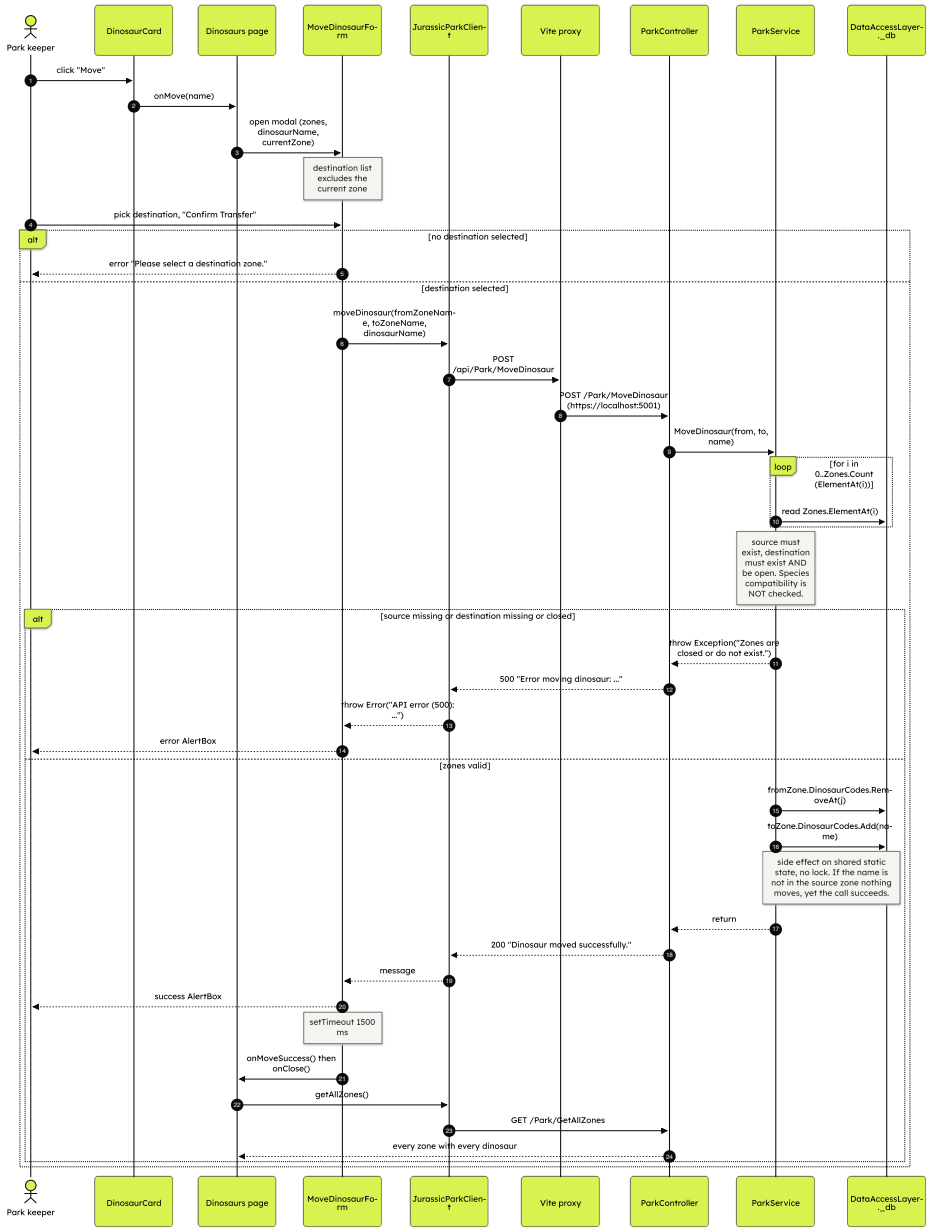
> **Entry points:** the "Move" button of `DinosaurCard.tsx:52`, rendered on the Dinosaurs and Zones pages · `POST /Park/MoveDinosaur` ( `ParkController.cs:58` ).

---

# Sequence

SEQUENCE · MOVE A DINOSAUR

RENDERED




# Step by step

**1** **UI trigger.** `DinosaurCard.tsx:52` calls `onMove(dinosaur.name)`; the page stores `{ name, zone }` in `selectedDinosaur` and renders `MoveDinosaurForm` (`Dinosaurs.tsx:253`).

**2** **Client-side validation.** `MoveDinosaurForm.tsx:27` removes the current zone from the options; `:36` rejects an empty destination. Closed zones are still listed.

| | |
|---|---|
| 3 | **Mapping to the request.** `MoveDinosaurForm.tsx:40` builds `MoveDinosaurRequest { fromZoneName, toZoneName, dinosaurName }` from component props, no ids. |
| 4 | **HTTP call.** `api.ts:70` posts JSON to `/api/Park/MoveDinosaur` with `credentials: 'include'`; `vite.config.ts` rewrites `/api` and forwards to `https://localhost:5001` with `secure: false`. |
| 5 | **Controller.** `ParkController.cs:58` forwards the three strings to the service; any exception becomes a `500` with the message in plain text, business errors included. |
| 6 | **Zone lookup.** `Class1.cs:74-84` scans the dictionary with `ElementAt(i)` in a `for` loop (quadratic) instead of `TryGetValue`. Source is matched by key without checking it is open; destination by key and `AccessStatus == true`. |
| 7 | **Mutation.** `Class1.cs:89-102` removes the code from the source list and adds it to the destination list, directly on `ZoneEntity` instances owned by the static store. The `DinosaurEntity` itself is untouched. |
| 8 | **Silent no-op.** If the dinosaur is not in the source zone, `dinosaur` stays `null` (`Class1.cs:99`), nothing moves, and the API still answers `200`. |
| 9 | **Refresh.** After a fixed 1.5 s delay (`MoveDinosaurForm.tsx:50`), the page reloads the whole park with `getAllZones()` (`Dinosaurs.tsx:74`). |

## Business rules

| Rule | Where enforced | Tested? |
|---|---|---|
| A destination must be selected | UI only, `MoveDinosaurForm.tsx:36` | NO |
| Destination differs from source | UI only, `MoveDinosaurForm.tsx:27` | NO |
| Destination exists and is open | `Class1.cs:80` | YES `Tests.cs:28` |
| Source zone exists | `Class1.cs:76` | NO |
| Dinosaur is in the source zone | Not enforced: silent no-op, `Class1.cs:99` | NO |

| Rule | Where enforced | Tested? |
| --- | --- | --- |
| Species can coexist in the destination | Not enforced: `CanSpeciesCoexist` exists but is never called on a move | NO |

---

## Risks on this flow CRITICAL

CRITICAL **No compatibility check.** A T-Rex can be moved into a Velociraptor or Triceratops zone; the park's main safety rule lives only on a separate "Compatibility" page.

CRITICAL **Unsynchronised mutation of shared static state.** Two concurrent moves can corrupt the `DinosaurCodes` lists.

WARNING **False success.** Moving a dinosaur that is not in the source zone returns `200`.

WARNING **Every business error is a `500`.** Clients cannot tell a closed zone from a crash.

WARNING **Over-fetching after each move.** The entire park is reloaded to reflect a change in two lists.

Product backlog

# Product backlog — jurassic-code

Analyzed 2026-03-09 · reverse-engineered from routes, services and tests · Skill by **Yoan Thirion**

AI-inferred from code — validate with product owners before use.

FEATURES

**9**

in 5 business domains

COVERAGE BY TESTS

**Features with a test** 6/9

One tested feature returns wrong data (feature 6)

INCOMPLETE OR DEFECTIVE

**4**

features 6, 7, 8 and 9

# Zone management

## 1. Create a park zone

TESTED

`POST /Park/AddZone · ParkService.AddZone · Class1.cs:27`

> *As a park operator, I want to create a new zone so that I can organize dinosaurs into separately managed areas.*

```
Scenario: Create an open zone
  Given the park has no zone named "Ismaloya Mountains"
  When I create a zone "Ismaloya Mountains" that is open
  Then the zone appears in the list of all zones
  And the zone is open

Scenario: Reject a duplicate zone
  Given a zone named "Western Ridge" already exists
  When I create another zone named "Western Ridge"
  Then the error "Zone already exists." is returned

Examples:
  | name          | isOpen | expected                  |
  | Main Temple   | true   | created                   |
  | Western Ridge | true   | error: already exists     |
  | Quarantine    | false  | created as a closed zone  |
```

## 2. Open or close a zone

TESTED

`POST /Park/ToggleZone · ParkService.ToggleZone · Class1.cs:110`

> *As a park safety officer, I want to toggle a zone between open and closed so that I can restrict dinosaur movements during incidents.*

```
Scenario Outline: Toggle a zone
  Given zone "<zone>" is <before>
  When I toggle zone "<zone>"
  Then zone "<zone>" is <after>

Examples:
  | zone          | before | after  |
  | Western Ridge | open   | closed |
  | Eastern Ridge | closed | open   |
```

GAP Toggling an unknown zone silently does nothing (`Class1.cs:112-119`).

### 3. List all zones

UNTESTED

`GET /Park/GetAllZones · ParkService.GetAllZones · Class1.cs:166`

> *As a park ranger, I want to see every zone with its dinosaurs so that I can monitor the park at a glance.*

```
Scenario: Retrieve all zones with their dinosaurs
  Given zones "Ismaloya Mountains", "Western Ridge" and "Eastern Ridge" exist
  And "Western Ridge" holds "Blue" and "Charlie"
  When I request all zones
  Then I receive 3 zones
  And "Western Ridge" lists "Blue" and "Charlie" with species, diet, health and last
```

## Dinosaur management

### 4. Add a dinosaur to a zone

TESTED

`POST /Park/AddDinosaurToZone · ParkService.AddDinosaurToZone · Class1.cs:41`

> *As a park keeper, I want to assign a dinosaur to an open zone so that I can track where each animal lives.*

```
Scenario: Add a dinosaur to an open zone
  Given zone "Ismaloya Mountains" is open
  When I add "Rexy" (T-Rex, carnivorous) to "Ismaloya Mountains"
  Then "Rexy" is listed in "Ismaloya Mountains"

Scenario: Reject a closed or unknown zone
  When I add "<dinosaur>" to "<zone>"
  Then the error "Zone is closed or does not exist." is returned

Examples:
  | dinosaur | zone       | zone state |
  | Stego    | Quarantine | closed     |
  | Dino     | Atlantis   | missing    |
```

GAP Adding a dinosaur whose name already exists throws a raw dictionary exception (`DataAccessLayer.cs:32`).

## 5. Move a dinosaur between zones

TESTED

`POST /Park/MoveDinosaur · ParkService.MoveDinosaur · Class1.cs:69 ·` **`detailed flow`**

> *As a park keeper, I want to move a dinosaur to another open zone so that I can respond to safety or habitat needs.*

```
Scenario: Move to an open zone
  Given "TestDino1" is in "Test Zone 1"
  And "Test Zone 2" is open
  When I move "TestDino1" from "Test Zone 1" to "Test Zone 2"
  Then "TestDino1" is in "Test Zone 2"
  And "TestDino1" is no longer in "Test Zone 1"

Scenario: Reject a closed or unknown destination
  When I move "TestDino1" from "Test Zone 1" to "<destination>"
  Then the error "Zones are closed or do not exist." is returned

Examples:
  | destination       | state   |
  | Test Zone 3       | closed  |
  | Non-Existent Zone | missing |
```

GAP Species compatibility is not checked, and a dinosaur absent from the source zone "moves" successfully without moving.

## 6. List dinosaurs in a zone

DEFECT

`POST /Park/GetDinosaursInZone · ParkService.GetDinosaursInZone · Class1.cs:141 · DataAccessLayer.cs:41`

> *As a park ranger, I want to see the dinosaurs of one zone so that I can check its population and health.*

```
Scenario: Retrieve the dinosaurs of a zone
  Given "Ismaloya Mountains" holds "Rexy" (T-Rex), "Bucky" (Triceratops) and "Echo" (
  When I request the dinosaurs of "Ismaloya Mountains"
  Then I receive 3 dinosaurs
  And "Bucky" has species "Triceratops" and is not carnivorous

Scenario: Unknown zone
  When I request the dinosaurs of "Ghost Zone"
  Then the error "Zone does not exist." is returned
```

**Defect.** `DataAccessLayer.GetDinosaurs` builds new entities holding only `CodeName`. Every dinosaur comes back with no species, flagged carnivorous, never sick, and fed on `0001-01-01`. The second "Then" fails today; the existing tests only count items.

# Safety

### 7. Check species coexistence

INCOMPLETE

`POST /Park/CanSpeciesCoexist · ParkService.CanSpeciesCoexist · Class1.cs:122`

> *As a park safety officer, I want to check whether two species can share a zone so that I can prevent attacks.*

```
Scenario Outline: Check a pair of species
  When I check whether "<species1>" and "<species2>" can coexist
  Then the answer is <expected>

Examples:
  | species1      | species2     | expected | covered by a test |
  | T-Rex         | Velociraptor | false    | yes               |
  | Triceratops   | Velociraptor | true     | yes               |
  | T-Rex         | Triceratops  | false    | no                |
  | Brachiosaurus | Stegosaurus  | true     | no (default)      |
```

GAP Only three species names are known; seed data uses "Tyrannosaurus Rex" (`Program.cs:37`), which the rule does not recognise.

---

# Health monitoring

### 8. Flag sick dinosaurs

UNTESTED

`Dinosaur.IsSick ↔ DinosaurEntity.HealthStatus · DinosaurCard.tsx`

> *As a park veterinarian, I want to identify sick dinosaurs so that I can prioritise care.*

```
Scenario: A sick dinosaur is flagged
  Given "Charlie" (Velociraptor) is sick and lives in "Western Ridge"
  When I list all zones
  Then "Charlie" is shown with "Medical Attention Required"
```

PARTIAL Stored and displayed only; no endpoint, alert or rule acts on it.

---

## Feeding

### 9. Track the last feeding time

■ UNTESTED

`Dinosaur.LastFed ↔ DinosaurEntity.FeedingTime`

> *As a park keeper, I want to see when each dinosaur was last fed so that I can schedule feeding rounds.*

```
Scenario: Feeding time is kept
  Given "Charlie" was last fed on 2026-03-08
  When I list all zones
  Then "Charlie" shows a last feeding time of 2026-03-08
```

■ PARTIAL Stored and mapped; no feeding endpoint or threshold.

Code quality

# Code quality — jurassic-code

Analyzed 2026-03-09 · backend C# focus · Skill by **Yoan Thirion**

---

OVERALL GRADE

F

## 14 smells

Small and readable, but built on global mutable state and broken encapsulation. Most smells sit in two files: `Class1.cs` and `Db2/`.

BY PRIORITY

| | |
|---|---|
| Critical | 5 |
| Significant | 5 |
| Minor | 4 |

RECOMMENDED STARTING POINT

**Global Data in DataAccessLayer**

Removing the static store unlocks DI, isolated tests, and fixes the reset bug.

## Defects spotted while reviewing ■ 3

Not smells but behaviour bugs, found by reading the code. Each one is invisible to the current tests.

■ CRITICAL **Data loss when listing a zone.** `DataAccessLayer.GetDinosaurs` (`DataAccessLayer.cs:41`) returns `new DinosaurEntity { CodeName = code }`: species, diet, health and feeding time are dropped, so `GetDinosaursInZone` reports every dinosaur as a carnivore of unknown species.

CRITICAL **Seed data wiped at first request.** `Program.InitializePark()` seeds four zones through `new ParkService()`; the DI singleton then runs the same constructor (`Class1.cs:23`), which calls `DataAccessLayer.Init()` and replaces the store with the three zones of `Init.cs`. "Main Temple" never reaches the UI.

WARNING **False success on move.** `MoveDinosaur` returns normally when the dinosaur is not in the source zone (`Class1.cs:99`). See the **flow**.

---

## Coupling / cohesion

### Global Data

CRITICAL

`Db2/DataAccessLayer.cs:7 · DataAccessLayer._db`

The whole park lives in `public static Database _db`. Every request, every test and every service instance reads and writes the same object, with no lock and no way to substitute it.

```
public static class DataAccessLayer
{
    public static Database _db;
    public static void Init() => _db = new Database();
```

**Refactoring:** Encapsulate Variable, then Replace Static with Instance: introduce an `IParkRepository` owning the dictionaries, register it as a singleton, inject it into `ParkService`.

### Mutable Data — constructor resets shared state

CRITICAL

`Class1.cs:21-25` · `ParkService()` · `Program.cs:24`

Creating a `ParkService` silently replaces the global store and re-seeds it. Instantiation order decides what data exists, which is the root of the "seed data wiped" defect.

```
public ParkService()
{
    DataAccessLayer.Init();
    Init();
}
```

**Refactoring:** Separate Query from Modifier: make the constructor side-effect free and move seeding to a dedicated, explicit startup step.

### Insider Trading

CRITICAL

`Db2/Database.cs:8-11` · `ReflectionHelper.cs:8`

`Database` declares `_dinosaurs` private, then hands it out through reflection on itself. Callers depend on a private field name that the compiler cannot check.

```
public static Dictionary<string, DinosaurEntity> Dinosaurs(Database db)
{
    return (Dictionary<string, DinosaurEntity>)ReflectionHelper.GetPrivateField(db, "
}
```

**Refactoring:** Expose the intent instead of the data: `AddDinosaur`, `FindDinosaur` methods on `Database` (or on the new repository), then delete `ReflectionHelper`.

### Mutable Data — service edits persistence entities

CRITICAL

`Class1.cs:94, 101, 116 · MoveDinosaur, ToggleZone`

`ParkService` reaches into `_db.Zones` and mutates `ZoneEntity.DinosaurCodes` and `AccessStatus` in place, bypassing `DataAccessLayer`'s own save methods. Persistence rules can no longer be enforced in one place.

**Refactoring:** Move Function: give the zone the behaviour (`zone.Toggle()`, `zone.Release(name)`, `zone.Admit(name)`) and persist through the repository.

---

## Data structures / modeling

### Duplicated Code — entity mapping

SIGNIFICANT

`Class1.cs:151-158 and 183-190 · GetDinosaursInZone, GetAllZones · plus the inverse in Class1.cs:48-55`

The `DinosaurEntity` → `Dinosaur` mapping, with its inverted booleans, is written twice; the reverse mapping a third time. A fix in one copy will be missed in the others.

**Refactoring:** Extract Function into a single mapper (`ToDomain` / `ToEntity`), covered by one test that pins every field.

### Primitive Obsession

SIGNIFICANT

`Class1.cs:53, 126-137 · CanSpeciesCoexist`

Zones and dinosaurs are identified by display names, species are compared to string literals, and health is the string `"Sick"` or `null`. "T-Rex" and "Tyrannosaurus Rex" are different species to the code.

**Refactoring:** Replace Primitive with Object: `ZoneName`, `Species` (with a diet), `HealthStatus` enum; drive compatibility from species data instead of literals.

---

## Size / complexity

### Deep Nesting

CRITICAL

`Class1.cs:29-37 · AddZone`

Four levels ( `foreach` › `if` › `if` › `if` ) to answer "does this key exist?". The `name != null` check is repeated on every iteration.

```
foreach (var zone in DataAccessLayer._db.Zones)
{ if (name != null)
    {
        if (zone.Value.ZoneCode != null)
        {
            if (zone.Value.ZoneCode == name) { throw new Exception("Zone already exis
```

**Refactoring:** Replace Nested Conditional with Guard Clauses, then Replace Loop with `Zones.ContainsKey(name)` .

### Loops

SIGNIFICANT

`Class1.cs:44-46, 74-84, 112-116 · AddDinosaurToZone, MoveDinosaur, ToggleZone`

Index loops call `Zones.ElementAt(i)` several times per iteration on a dictionary: quadratic lookups where a keyed access is constant time, and the intent is buried.

**Refactoring:** Replace Loop with Pipeline, or simply `TryGetValue` on the key.

### Duplicated Code — controller error handling

SIGNIFICANT

`JurassicCode.API/Controllers/ParkController.cs:24, 38, 52, 66, 80, 94, 108`

Seven identical `catch (Exception)` blocks map every failure to `500` . Changing the error contract means editing seven places (Divergent Change).

**Refactoring:** Extract the policy into exception middleware or an `IExceptionFilter` ; throw domain exceptions mapped to `400` / `404` / `409` .

### Large Class — split by `partial`

SIGNIFICANT

`Class1.cs (198 lines) + Init.cs (277 lines) · ParkService`

The service mixes use cases and 277 lines of fixture data; `partial` hides the size without separating responsibilities.

**Refactoring:** Extract Class: a `ParkSeeder` for the data, keep `ParkService` for use cases.

---

## Readability / naming

### Mysterious Name

MINOR

`Class1.cs · namespace Db2 · project JurrassicCode.Console · DinosaurEntity.IsVegan`

A default IDE file name for the central service, a versioned namespace, a typo in a project name, and `IsVegan` as the negation of `IsCarnivorous`.

**Refactoring:** Rename File / Rename Field: `ParkService.cs`, `Persistence`, `JurassicCode.Console`, `IsCarnivorous` on both sides.

### Comments

MINOR

`Class1.cs:124 · Startup.cs:20, 35, 47, 57`

"Logique complexe et inutile pour déterminer la coexistence" apologises for the code instead of fixing it; the Startup comments are template boilerplate.

**Refactoring:** Extract Function with an intention-revealing name, delete the comments.

### Lazy Element

MINOR

`JurassicCode.API/Controllers/WeatherForecastController.cs · WeatherForecast.cs`

The ASP.NET template's weather endpoint is still compiled and exposed in Swagger.

**Refactoring:** Remove Dead Code.

### Speculative Generality

■ MINOR

`JurassicCode.csproj · Confluent.Kafka 0.11.1, Polly 2.2.0`

Messaging and resilience libraries declared for features that do not exist.

**Refactoring:** Remove the package references; reintroduce current versions when a real need appears.

---

## Summary

14 smells detected: 5 critical, 5 significant, 4 minor, plus 3 behaviour defects.

**Recommended starting point: Global Data in `DataAccessLayer`.** Replacing the static store with an injected repository removes the constructor reset (and its seed-data bug), makes `ParkService` testable in isolation, and gives the reflection hack and the direct entity mutations a proper home. Pin the current behaviour first with tests that assert returned fields, which will immediately expose the `GetDinosaurs` data loss.